

37. Webová uživatelská a aplikační rozhraní, správa sezení a autentizace

37. Webová rozhraní, sezení a autentizace

SZZ okruh 37 (FIT BUT). Klientská i serverová strana webu + stav a identita uživatele.

Shrnutí

Webová uživatelská rozhraní

- **HTML** (struktura, značky), **CSS** (vzhled, oddělení od obsahu), **JavaScript** (dynamika; manipule **DOM**, **AJAX** pro asynchronní dotazy, event listeners).

Aplikační rozhraní (API)

- **SOAP** — standardizovaný protokol, XML (objemné, nezávislé na HTTP).
- **REST** — architektura nad HTTP; CRUD □ POST/GET/PUT-PATCH/DELETE; bezstavové, většinou JSON.
- **GraphQL** — dotazovací jazyk; klient si vyžádá přesně potřebná data; JSON.

Správa sezení a autentizace

- **HTTP je bezstavový** □ stav přes **session ID** předávané v **cookie**. Sezení ≠ autentizace.
- **Autentizace** (ověření identity): HTTP **Authorization** hlavička (Basic/Bearer, vyžaduje HTTPS), formulář + cookie, **JWT** (header.payload.signature, podepsané, čitelné), **OAuth** (přístup k cizí službě bez sdílení hesla, Access Token), **SSO** (jedno přihlášení, Kerberos/LDAP).
- Pozor: **autentizace** = ověření identity × **autorizace** = povolení přístupu/operace.

Relační DB v pozadí viz [[okruhy/36-relacni-data-sql-transakce]]; principy GUI a MVC viz [[okruhy/13-graficka-uzivatelska-rozhrani]]; kryptografie pro JWT/TLS viz [[okruhy/44-smerovani-zabezpeceni-siti]]; HTTP jako služba viz [[okruhy/42-sluzby-aplikacni-vrstvy]].

Související syntéza

- [[synthesis/kryptografie-autentizace | Kryptografie × autentizace]] — syntéza

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Sezení a autentizace □ [[#Správa sezení a autentizace]] - Proč session, když je HTTP bezstavový? □ Stav se udrží přes session ID v cookie, klient ho posílá s každým dotazem. - Session vs. JWT? □ Session = stav na serveru (ID v cookie); JWT = podepsaný token nesený klientem (stav u klienta). - Autentizace

pomocí session? □ jméno+heslo □ ověření □ session + ID do cookie □ každý dotaz ověřen. - *OAuth?* □ Přístup aplikace k cizí službě bez sdílení hesla (Access Token).

REST a HTTP □ [[#Aplikační rozhraní (API)]] - *REST metody?* □ GET (read), POST (create), PUT/PATCH (update), DELETE; bezstavové, resource-oriented. - *SOAP vs. REST vs. GraphQL?* □ SOAP standardizovaný XML protokol; REST jednoduchá HTTP architektura (JSON); GraphQL dotaz na přesná data.

Webová UI □ [[#Webová uživatelská rozhraní]] - *HTML/CSS/JS role?* □ struktura / vzhled / dynamika (DOM, AJAX).

Plné znění (ke studiu)

Webová uživatelská rozhraní

Webová uživatelská rozhraní jsou založená na technologiích HTML, CSS, a JavaScript, se kterými dokáží pracovat webové prohlížeče.

HTML

Hypertext Markup Language je značkovací jazyk používaný pro tvorbu webových stránek, které jsou propojeny **hypertextovými odkazy**.

- MIME: **text/html**,
- koncovka: **.html**, **.htm**

Základem HTML jsou **značky**, které vymezují **úseky** dokumentu. Obvykle jsou značky **párové** (<div></div>, <p></p>), ale existují i **nepárové** (prázdné - void,
, <hr>), tyto značky mohou mít pouze atributy, ale nemají obsah. Dále HTML dokument tvoří **komentáře/poznámky**, které nezpracovává prohlížeč (<!-- Libovolný text -->). Pro vkládání **speciálních znaků** (<, >, &, ...) používáme escape sekvence (< > & ...). Používáním HTML značek vytváříme **stromovou strukturu** dokumentu.

Samotný html obsah bez stylování pomocí CSS není uživatelsky přívětivý (má dle mého názoru škaredý vzhled), proto se zavedlo CSS na úpravu vzhledu HTML dokumentu.. [[media/szz-37/media/image1.png]]

[[media/szz-37/media/image2.png]]

Základní ovládací prvky HTML

- **<input>**: slouží pro zadání vstupu uživatelem, který poté lze zaslat na server pomocí **formuláře** metodou POST nebo GET. Má různé typy (text, submit, phone, hidden, button, email, radio, ...)
- **<textarea></textarea>**: slouží pro zadání rozsáhlého textu, lze nastavit kolik má mít řádků a sloupců a může být zvětšovací.
- **<select><option></option></select>**: umožňuje uživateli výběr jedné z hodnot.
- **<button></button>**: tlačítko například pro odeslání formuláře.

Základní HTML prvky pro zobrazování obsahu

- **<nav></nav>**: element, který by měl obsahovat navigaci na stránce pro SEO,
- ****: očíslovaný seznam,
- ****: neočíslovaný seznam,

- **<table><tr><td></td></tr></table>**: tabulka (zvýraznění buněk pomocí <th>), lze nastavit spojování buněk,
- **<a href>**: odkaz na jinou stránku,
- **<p></p>**: odstavec,
- **<div></div>**: obecný blokový element,
- **<div>**: obecný řádkový element.

Kaskádové styly (CSS, Cascading Style Sheets)

Mechanismus (jazyk), který umožňuje návrhářům oddělit vzhled dokumentu od jeho struktury a obsahu. HTML elementům lze pomocí atributu **class** přidat různé způsoby zobrazení, element lze také stylovat pomocí elementu **style**, to ale není doporučeno. Pomocí kaskádových stylů lze:

- **stylovat text**: barvu, velikost, font, podtržení, kurzívu, tučný text, aj.
- **nastavovat odsazení elementů**,
- **nastavovat pozici elementů**,
- **specifikovat barvu**: pozadí, okraje, stínu,
- **vytvářet animace**,
- **specifikovat různé zobrazení pro různé velké obrazovky**,
- **specifikovat zobrazení pro tisk**.

JavaScript JS

JS je **skriptovací objektově orientovaný jazyk**, který **dokáže interpretovat** (snad) všechny **webové prohlížeče** a umožňuje tak vytvářet **dynamické webové stránky**. Lze pomocí něj manipulovat s **DOM** (Document Object Model), což je objektově orientovaná reprezentace HTML (a XML). Lze tak měnit vzhled elementů a přidávat nebo odebírat je bez znovunačtení stránky. Současně lze pomocí JS **asynchronně** (tj. bez toho aby se zaseklo uživatelské rozhraní) dotazovat server pomocí **AJAX** (Asynchronous JavaScript And XML) a umožnit tak například zapsání změn bez nutnosti načtení stránky, jinak řečeno umožňuje tvořit jednostránkové aplikace. JS na HTML elementy napojujeme pomocí **DOM interface** (jedno z WEB APIs). Pomocí **event listeners** zachytáváme události, které mohou být posílané z HTML stránky, například při stlačení tlačítka, scrollování, ale i při načítání stránky.

Aplikační rozhraní

Aplikační rozhraní nabízí způsob, jak spolu mohou **komunikovat** dvě (a více) aplikací, respektive jedna aplikace se může **dotazovat** na informace, které poskytuje ta druhá. Typickým příkladem využití webového API je, když vytvářím aplikaci, kde musí uživatel vyplnit adresu (např. nějaká dovožková služba), použiji API, které validuje adresu a umí na základě ulice vyhledat město, zemi, PSČ atd. Uživateli lze nabídnout tento výsledek dotazu na API a on se nemusí namáhat s vyplňováním a neudělá omylem chybu. Existuje několik způsobů, jak lze API programovat.

SOAP – Simple Object Access Protocol

Standardizovaný protokol, který specifikuje, strukturu dotazu i odpovědi na dotaz. Je nezávislý na HTTP, lze jej použít i např. s SMTP protokolem. Každá zpráva je zabalena do **obálky (envelope)** a je tvořena **hlavičkou (header)**, která však není povinná a ve zprávě může chybět, a **tělem (body)** zprávy, které obsahuje dotaz, respektive odpověď na dotaz. Pro serializaci dat používá formát XML.

- **výhody:** jedná se o standardizovaný protokol (vhodnější pro veřejná api, např. vládní)
- **nevýhody:** objemnost přenášených dat, která je dána použitím serializačního formátu XML. S tím také souvisí náročné zpracování a validace příchozích dat.

REST – Representational State Transfer

REST není protokol, jedná se pouze o návrh architektury, který by měly splňovat systémy, které chtějí implementovat tak zvané RESTful aplikační rozhraní. Je závislý na HTTP protokolu a využívá jeho metody a stavové kódy. Nad každým zdrojem jsou definovány CRUD operace a využití metod je následující:

- **C** zastává operaci **Create** – vytvoření objektu/objektů daného typu, operace se váže na HTTP dotaz typu **POST**,
- **R** zastává operaci **Read** – čtení objektu/objektů, operace se váže na HTTP dotaz typu **GET**,
- **U** zastává operaci **Update** – aktualizaci objektu/objektů, operace se váže na HTTP dotaz typu **PUT** nebo **PATCH**,
- **D** zastává operaci **Delete** – odstranění objektu/objektů, operace se váže na HTTP dotaz typu **DELETE**.

Jako serializační formát REST používá nejčastěji JSON, ale lze použít i XML či jiný. Zhodnocení REST:

- **výhody:** jednoduchost implementace a široká podpora různými knihovnamy a frameworky. Integrace s protokolem HTTP a z toho plynoucí jednodušší zpracování dotazů a také flexibilita při výběru formátu přenášených dat.
- **nevýhody:** nejedná o standard. To způsobuje odlišnosti u jednotlivých implementací, ať už jde o použitý formát dat, nebo i způsob sestavování URL koncových bodů.

GraphQL – Graph Query Language

GraphQL také není protokol, jedná se o silně typovaný jazyk definující syntaxi, pomocí níž lze vytvořit dotaz žádající přesně konkrétní data. Také není závislý na protokolu a kromě HTTP lze využít i s např. MQTT. Pro serializaci dat používá GQL výhradně formát JSON.

- **výhody:** Hlavní výhodou aplikačního rozhraní implementovaného pomocí GraphQL je, že server odesílá v odpovědi pouze na data, o která si klient v dotazu žádá.
- **nevýhody:** malá rozšířenost, komplexní dotazy mohou představovat příliš velkou zátěž na server. Dotazy nelze cachovat, protože se mění.

Správa sezení

Posloupnost dotazů (a odpovědí na ně) jednoho uživatele od okamžiku navštívení stránky až po její opuštění. Řeší se přiřazením **unikátního identifikátoru** (Session ID) uživateli při prvním navštívení stránky. Sezení **neřeší autentizaci**, pouze rozlišuje dotazy jednotlivých uživatelů (např. uživatel si v rámci sezení může nějak upravit chování stránky, např. dark mode, a stránka se tak chová do konce sezení). Protože je protokol **HTTP bezstavový** (není zachována žádná informace mezi jednotlivými dotazy) musí klient Session ID pokaždé zasílat při dotazu. To lze realizovat zasíláním v rámci dotazu, např. jako součást URL, což je dost nepraktické. Proto se identifikátor sezení **ukládá do Cookies**. Cookie je malý objem dat, který si server může uložit na klientovi a ten je poté zasílá při každém dotazu. Odcizení Session ID může být potenciálně nebezpečné.

Autentizace

Autentizace je **proces ověření identity uživatele** (nebo jiného objektu). Autentizace vůči webovým službám je typicky řešena nějakou **tajnou informací**, kterou může znát pouze uživatel, který se autentizuje (typicky uživatelské jméno a heslo). Možnosti autentizace u webových aplikací:

- **Authorization hlavička protokolu HTTP:** Jedná se o mechanismus vestavěný pro **autentizaci** (i když název hlavičky tomu **neodpovídá***) do protokolu HTTP. **Vyžaduje** použití **HTTPS**. Používá se spíše při autentizaci vůči API, protože je nutné, aby každý dotaz obsahoval tuto hlavičku. Předávaná data mohou mít více formátů např. **Basic** a **Bearer** (údaje se předávají kódované v Base64). Při GET dotazech lze zadat údaje v url: `http://username:password@example.com/`
 - ***Autorizace** je **proces získávání souhlasu s provedením nějaké operace, povolení přístupu** někam nebo něčemu (přístup k informacím, funkcím, programovým objektům a podobně).
- **Autentizace pomocí HTML formuláře a Cookies:** Další možností autentizace je odeslat autentizační údaje v rámci POST metody na server, ten je zpracuje a vyhodnotí, vytvoří uživateli Cookie indikující, že je autentizován (Cookie je obvykle nečitelná - nějakým způsobem šifrovaná) a klient pak při každém dotazu tuto Cookie odesílá na server.
- **JSON Web Token (JWT):** Složen ze tří částí
 - **Header** (hlavička) – obsahující účel a použité algoritmy,
 - **Payload** (obsah) – JSON data obsahující informace o uživateli,
 - **Signature** (podpis) – informace pro ověření, že token nebyl podvržen nebo změněn cestou. Jedná se o **podepsanou hlavičku a obsah**, hlavička a obsah ale zůstávají **čitelné**. Používá se asymetrická kryptografie (případně symetrická, pokud je server současně i autentizačním serverem).

Tyto 3 části se kódují se pomocí **base64**. Většinou se posílají v hlavičce **Authorization: Bearer base64(xxxxx).base64(yyyyy).base64(zzzzz)**. Postup autentizace:

1. Klient kontaktuje **autentizační server** a **dodá autentizační údaje** (autentizační server může být server používané aplikace nebo nějaký jiný, např. Twitter).
 2. Autentizační server vygeneruje **podepsaný JWT token** a vrátí jej klientovi.
 3. Klient jej zasílá při každém dotazu na server nebo mu jsou vytvořeny nějaké jiné autentizační údaje, např. pomocí cookies.
 4. Server získá public key od autentizačního serveru a zkontroluje, že podpis odpovídá.
- **OAuth:** OAuth (Open Authorization) řeší problém, kdy chce uživatel **umožnit aplikaci přístup** k nějaké **webové službě** s API, u které má účet, ale **nechce aplikaci sdělit své přihlašovací údaje** k této službě. (Příkladem je např. souhlas, že Draw.io může přistupovat ke Google Disk). Realizuje se tak, že daná aplikace **přesměruje** uživatele na **stránky webové služby**, kterou požaduje. Tam uživatel **potvrdí, že důvěřuje** této aplikaci a daná webová služba vygeneruje tzv. **Access Token**, pomocí kterého poté provádí dotazy na API webové služby.
 - **Single Sign On (SSO):** Autentizační mechanismus, který umožňuje uživateli přihlásit se pomocí jediného ID a hesla do několika souvisejících, ale nezávislých systémů. Využívá k tomu **adresářové služby** (LDAP, Active Directory) a protokoly jako **NTLM** a **Kerberos**. Uživateli se pak stačí přihlásit do adresářové služby (při přihlášení na PC např. do Windows) a pomocí jmenovaných protokolů se poté provádí autentizace vůči daným webovým službám za přihlášeného uživatele na PC.

Zdroje

- SZZ okruh 37 — studijní materiály FIT BUT (szz-37.docx). Obrázky: media/szz-37/.

Navigace: [[index | • Přehled okruhů]] · □ [[okruhy/36-relacni-data-sql-transakce | 36. Strukturovaná data, SQL, transakce]] · další: [[okruhy/38-sprava-souboru-pameti | 38. Správa souborů a paměti]] □